

# Estructura de Computadores

Segundo curso



# Introducción a Verilog

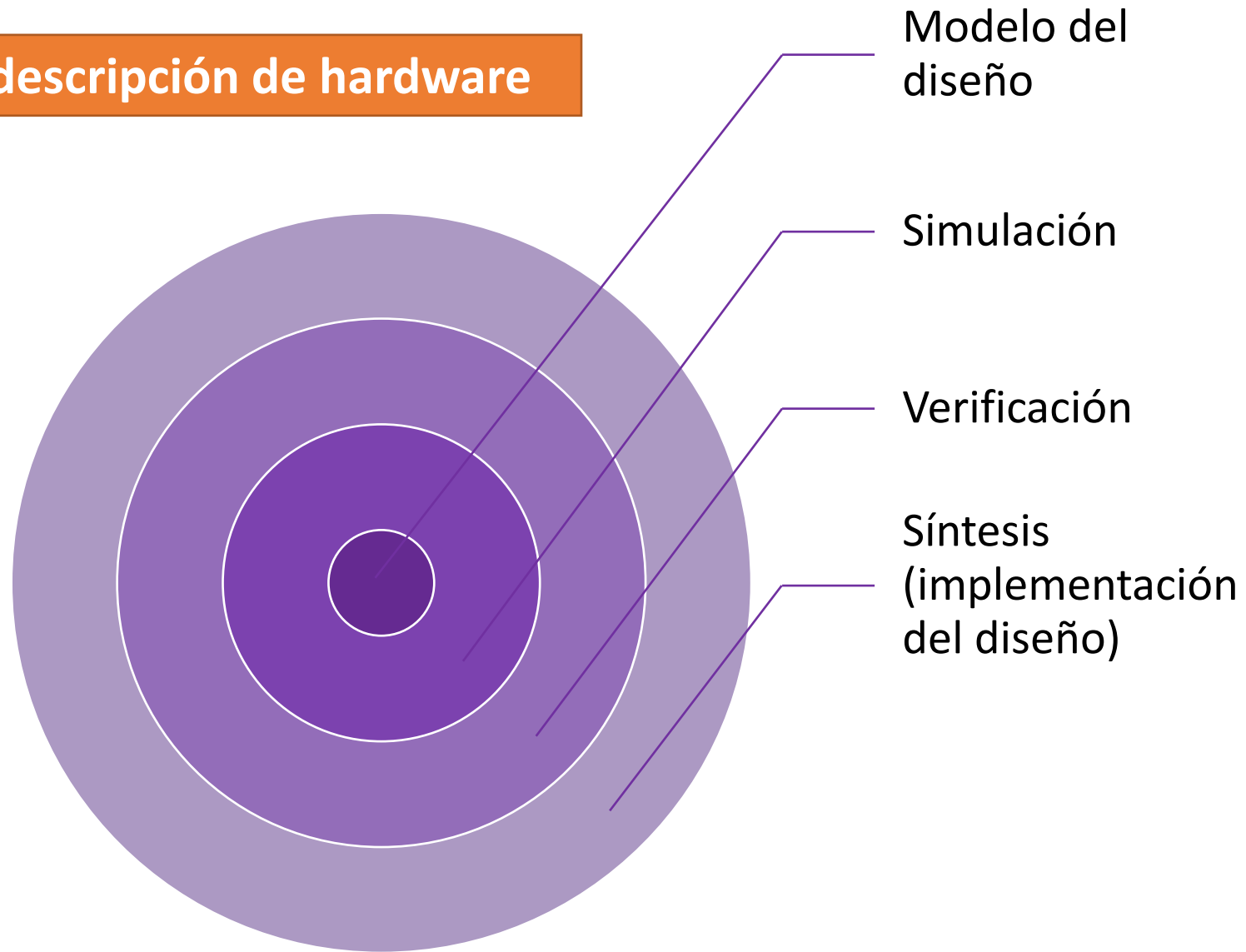


Verilog **NO ES** (estrictamente hablando) un **lenguaje de programación**

- Aunque permite indicar el comportamiento de un dispositivo hardware mediante programación



## Lenguaje de descripción de hardware



# Lenguaje C

- Distingue mayúsculas y minúsculas
- Sintaxis similar (comentarios y demás)



# Conductual (behavioral)

Se indica **qué hace** el módulo

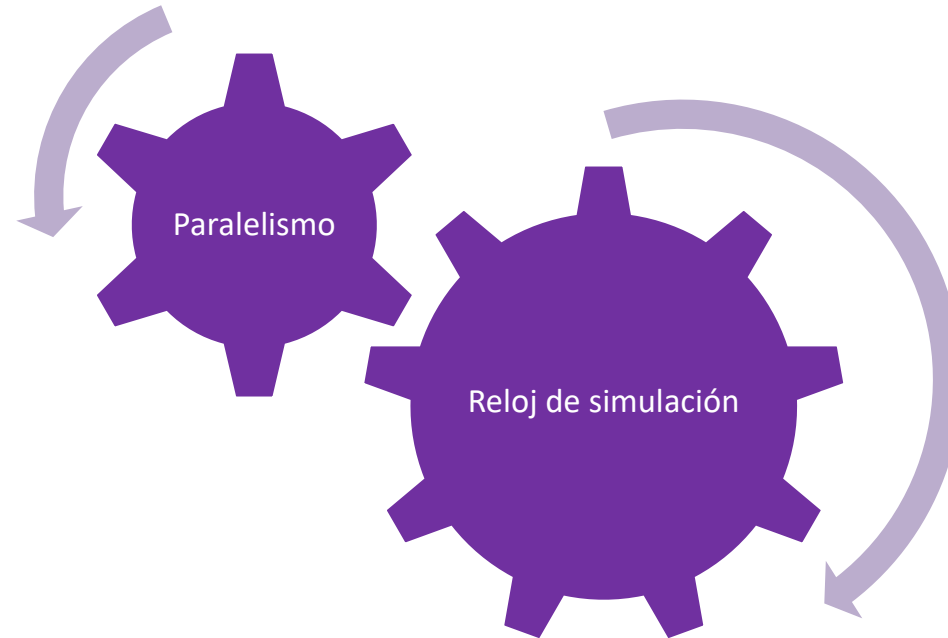
# Estructural

Se indican **los elementos que componen** el módulo



- Incluso en un modelado conductual...

**Verilog no define un comportamiento secuencial al uso**



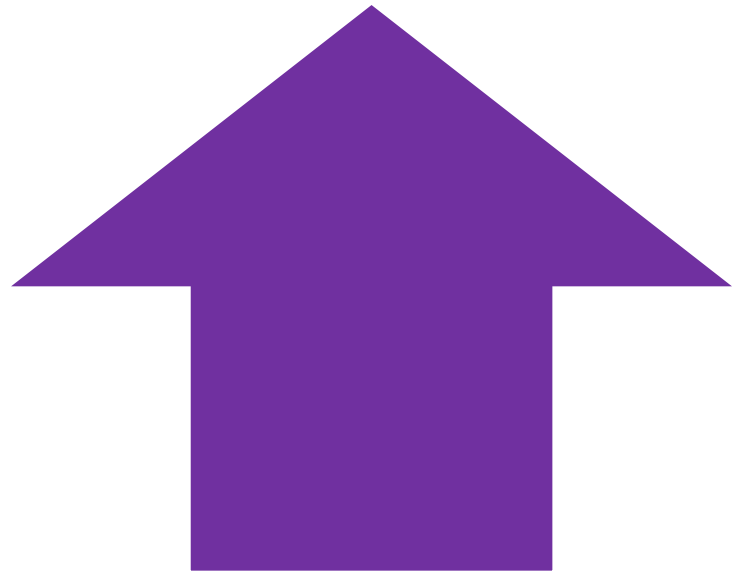
Flexibilidad

Diseño top-down

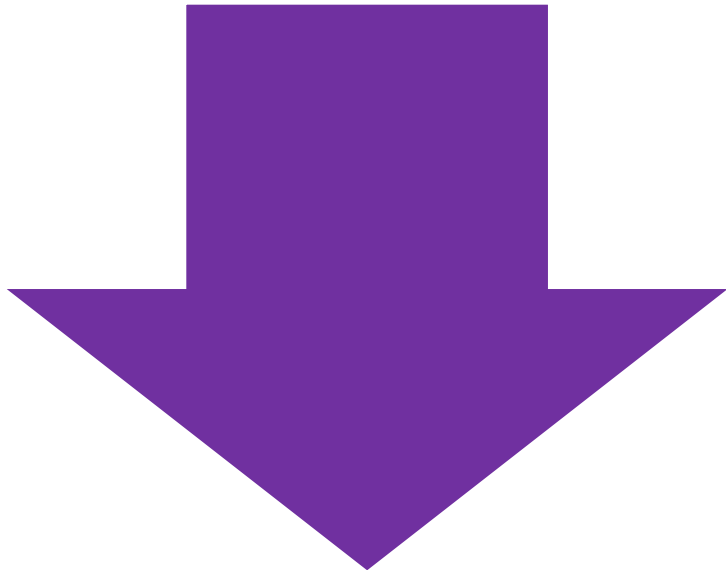
- Hasta el nivel del transistor

Potencia del  
software de  
síntesis



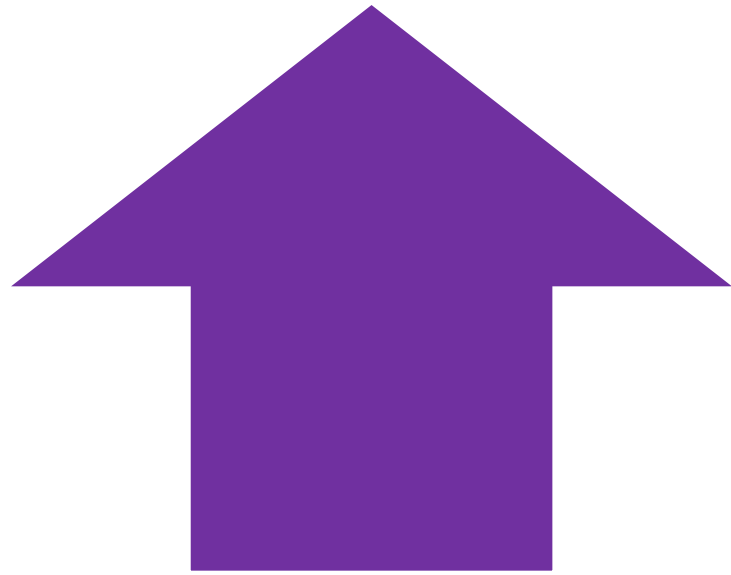


Sintetizable

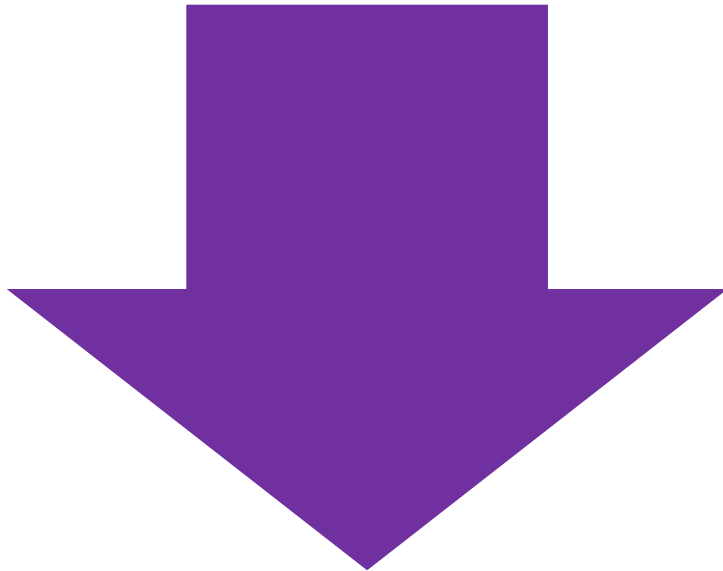


No  
synthesizable





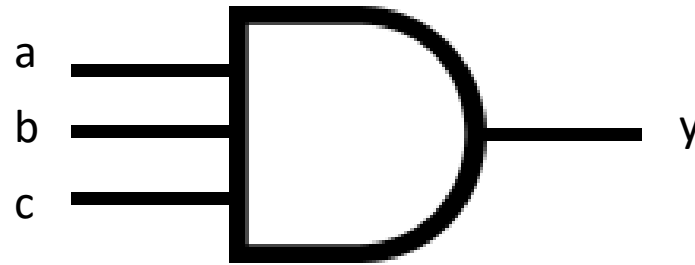
Módulos de interés  
Sintetizable



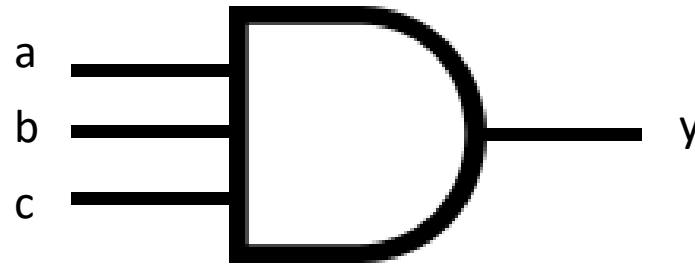
Módulos de testeo  
No sintetizable



- Bloque o componente hardware, con sus entradas y salidas



- Bloque o componente hardware, con sus entradas y salidas



```
module puerta_and(output y, input a, b, c);  
    assign y = a & b & c;  
endmodule
```

assign: asignación continua



| Valor | Significado                                                                                         |
|-------|-----------------------------------------------------------------------------------------------------|
| 0     | Representa el cero lógico o una condición falsa                                                     |
| 1     | Representa el uno lógico, o una condición verdadera                                                 |
| x     | Representa un valor lógico desconocido. Al inicio de la simulación las variables adoptan este valor |
| z     | Representa un estado de alta impedancia                                                             |



## nets

- Conexión entre módulos
- No permiten almacenar ningún valor
- Ejemplo: **wire**/tri (funciona como una “or cableada”)

## registers

- “Variables”: pueden almacenar el valor
- Ejemplos: **reg**, integer...



| Valor de wire/tri | 0 | 1 | x | z |
|-------------------|---|---|---|---|
| 0                 | 0 | x | x | 0 |
| 1                 | X | 1 | x | 1 |
| X                 | x | x | x | x |
| Z                 | 0 | 1 | x | z |

Input: entrada  
a un módulo

- Siempre tipo “wire”

Output: salida  
de un módulo

- Pueden ser “reg” o “wire”
- Sólo “de puertas adentro”

También puede usarse dentro del módulo



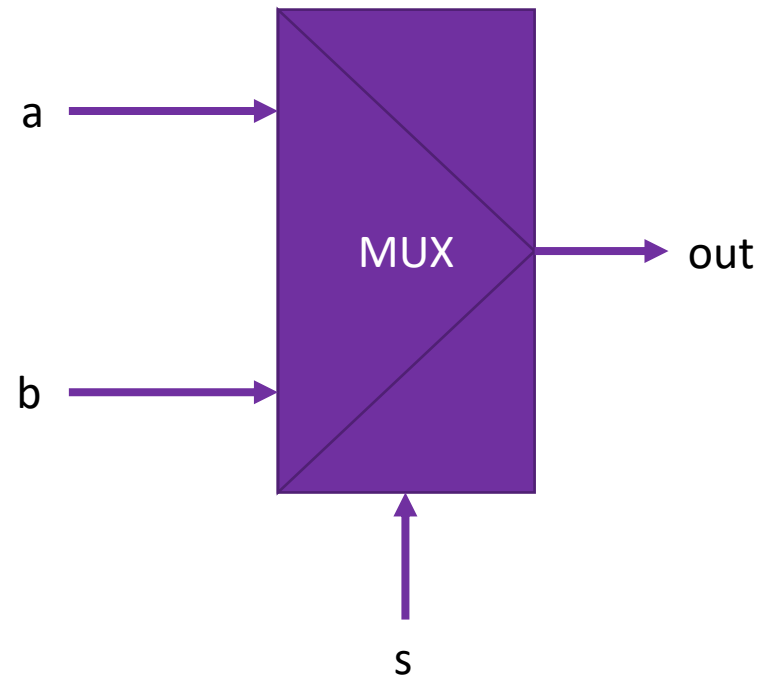
`Nombre_Modulo nombre_instancia(puertos_módulo)`

Ejemplo: podemos instanciar la puerta and que creamos antes

```
puerta_and mi_puerta_and(y, a, b, c);
```

¡Esto NO es una llamada a una función!





- <tamaño>'<base><valor>

- Ejemplos:

| Constante  | Almacenado como                  |
|------------|----------------------------------|
| 1          | 00000000000000000000000000000001 |
| 8'hAA      | 10101010                         |
| 6'b10_0011 | 100011                           |
| 'hF        | 00000000000000000000000000001111 |
| 6'hCA      | 001010                           |
| 6'hA       | 001010                           |
| 16'bz      | zzzzzzzzzzzzzzzzzz               |
| 8'bx       | xxxxxxxx                         |
| 8'b1       | 00000001                         |

```
wire [n:m] nombre_array;
```

Define un “bus” de ancho  $(n - m + 1)$

Ejemplo:

```
wire [7:0] CABLE_GORDO; // Ancho = 7 - 0 + 1 = 8 bits
```

```
reg [n:m] nombre_array [p1:p0];
```

Define una “memoria” que contiene  $(p1 - p0 + 1)$  palabras de  $(n - m + 1)$  bits

Ejemplo:

```
reg [7:0] MEMORIA [127:0]; // 128 palabras de 8 bits cada una
```

¡Ojo! “n” no tiene que ser mayor que “m”, ni “m” ser 0 siempre



## initial

- Se ejecuta una única vez
- Empieza en el instante 0 de la simulación
- No es sintetizable

## always

- Se ejecuta como si fuera un bucle
- Síncrono (basado en tiempo) o asíncrono (basado en eventos)
- Los eventos son cambios en otras señales del sistema → Lista sensible



- always @(lista\_sensible)

```
always @signal1 out=c+d
// Realiza la asignación cada vez que cambia el valor de signal1
always @(b or c) out=b&d
// Realiza la asignación cada vez que cambian b o c
always (posedge clk1 or negedge clk2) out=0
/* Realiza la asignación ante un flanco de subida en clk1 o un flanco de
bajada en clk2 */
```



- Las asignaciones que se realizan dentro de un proceso (procedurales) no usan la palabra reservada “assign”
- Si hay más de una asignación procedural dentro de un proceso

```
begin
```

```
...
```

```
end
```



- Procedurales

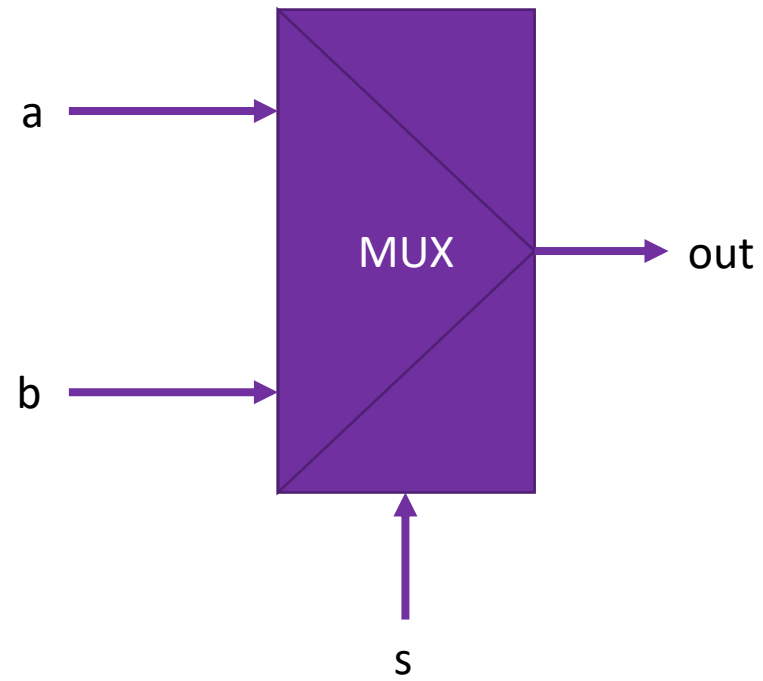
- Bloqueantes: “=”
  - `a = 1'b1;`
- No bloqueantes: “<=”
  - `a <= 1'b1;`
- **Siempre a reg**

Veremos la diferencia en la previa a la práctica 2

- Continuas

- `assign a = 1'b1;`
- **Siempre a wire**





- Se indica como

#<t>

donde <t> es el tiempo del retardo

- Son relativos al instante actual de simulación
- La unidad de tiempo se puede indicar con una directiva

```
`timescale unidad_tiempo / precisión  
donde unidad_tiempo ≥ precisión
```

- Ejemplo

```
`timescale 1ns / 10ps
```

```
#3; // Correspondería a un retardo de 3 ns desde el instante actual
```



# Banco de pruebas

- fichero especial que usamos para probar nuestros diseños
- Generalmente sin entradas ni salidas
- Define:
  - Módulo a verificar (módulo *dut*)
  - Señales de entrada para el *dut* y análisis de sus salidas



## \$finish

- Indica el fin de la simulación
- Si no lo ponemos, nunca termina

## \$display, \$write

- Equivalentes al printf (display añade saltos de línea)

## \$monitor

- Como el printf, pero lo indicamos una única vez
- Imprime por pantalla cada vez que cambia cualquiera de las variables indicadas



# \$dumpfile y \$dumpvars

- Para obtener las trazas que visualizamos con el gtkwave

```
$dumpfile("nombre_de_fichero.vcd");  
$dumpvars(lista de variables que deseamos guardar  
separadas por comas);  
// $dumpvars; guarda todas
```



# Operadores

- Páginas 7-8 tutorial Jorge Chávez

# Estructuras de control

- Páginas 8-9 tutorial Jorge Chávez



## El operador concatenación

$$\{cout, s\} = a + b + cin;$$
